



INGENIERÍA DE SOFTWARE

Predictor KRL

Sistema de Cálculo del
Coeficiente de Refracción Local



IS

Tabla de Contenidos

1. Información del Proyecto	1
1.1. Identificación del Proyecto	1
1.2. Equipo de Desarrollo	1
2. Metodología de Desarrollo	1
2.1. Enfoque Metodológico	1
2.2. Principios Aplicados	1
2.3. Ciclo de Desarrollo	2
3. Análisis y Diseño del Sistema	3
3.1. Descripción General	3
3.2. Requerimientos Funcionales	3
3.3. Requerimientos No Funcionales	3
3.4. Casos de Uso	4
4. Arquitectura del Sistema	4
4.1. Arquitectura General	4
4.2. Stack Tecnológico	5
4.3. Componentes del Sistema	6
5. Detalles de Implementación	6
5.1. Fórmula del Coeficiente KRL	6
5.2. Validación de Datos	6
5.3. Medidor Visual	7
6. Pruebas y Validación	7
6.1. Estrategia de Pruebas	7
6.2. Casos de Prueba	7
7. Instalación y Despliegue	8
7.1. Requisitos del Sistema	8
7.2. Proceso de Instalación	8
8. Mantenimiento y Soporte	8
Plan de Mantenimiento	8
9. Conclusiones	8

Tabla de Figuras

<i>Tabla 1 : Identificación del proyecto</i>	1
<i>Tabla 2 : Distribución del equipo de desarrollo</i>	1
<i>Tabla 3 : Fases, actividades y entregables del ciclo de desarrollo</i>	2
<i>Tabla 4 : CU-01: Calcular Coeficiente KRL</i>	4
<i>Tabla 5 : Distribución de responsabilidades</i>	5
<i>Tabla 6 : Stack tecnológico</i>	5
<i>Tabla 7 : Casos de prueba</i>	7

1. Información del Proyecto

1.1. Identificación del Proyecto

Nombre del Sistema	Predictor KRL
Versión	1.0
Fecha	Febrero 2026
Metodología	Desarrollo Ágil Adaptado (XP Simplificado)

Tabla 1: Identificación del proyecto

1.2. Equipo de Desarrollo

Rol	Responsable	Responsabilidades
Experto de Dominio / Product Owner	[Creador de la Fórmula]	Validación científica, definición de requerimientos, especificación de la fórmula KRL
Desarrollador Implementador	[Ingeniero Informático]	Diseño e implementación del software, interfaz gráfica, pruebas, documentación técnica

Tabla 2: Distribución del equipo de desarrollo

2. Metodología de Desarrollo

2.1. Enfoque Metodológico

Dado el contexto de un equipo reducido compuesto por el creador de la fórmula científica y el implementador técnico, se ha adoptado un enfoque ágil simplificado basado en Extreme Programming (XP). Esta metodología es ideal para proyectos pequeños donde la comunicación directa y la retroalimentación continua son esenciales.

2.2. Principios Aplicados

- Comunicación directa: Interacción constante entre el experto de dominio y el desarrollador para validar la implementación de la fórmula KRL.

- Simplicidad: Diseño minimalista centrado en la funcionalidad esencial del cálculo del coeficiente de refracción.
- Retroalimentación inmediata: Validación científica continua de los resultados mediante pruebas con datos reales.
- Iteraciones cortas: Desarrollo incremental con entregas funcionales frecuentes.
- Código limpio: Implementación clara y bien documentada que facilita el mantenimiento futuro.

2.3. Ciclo de Desarrollo

Fase	Actividades	Entregables
Planificación	Definición de requerimientos, especificación de la fórmula KRL, diseño de interfaz	Documento de especificaciones, mockups de interfaz
Diseño	Arquitectura del sistema, selección de tecnologías, diseño de componentes	Diagrama de arquitectura, esquemas de datos
Implementación	Codificación de la interfaz gráfica, implementación del algoritmo KRL, integración de componentes	Código fuente funcional, interfaz operativa
Pruebas	Validación científica de cálculos, pruebas de interfaz, casos de prueba con datos reales	Reporte de pruebas, validación de resultados
Documentación	Elaboración de manuales, documentación técnica, guías de usuario	Manual de usuario, documento de ingeniería

Tabla 3: Fases, actividades y entregables del ciclo de desarrollo

3. Análisis y Diseño del Sistema

3.1. Descripción General

El Predictor KRL es una aplicación de escritorio diseñada para calcular el Coeficiente de Refracción Local (KRL), un parámetro fundamental en estudios atmosféricos y geodésicos. El sistema permite a los usuarios ingresar variables meteorológicas y obtener el valor del coeficiente de manera rápida y precisa, con una representación visual intuitiva del resultado.

3.2. Requerimientos Funcionales

- RF-01: El sistema debe permitir el ingreso de cuatro parámetros: altura (m), temperatura (°C), presión (hPa) y gradiente vertical de temperatura (°C/m).
- RF-02: El sistema debe calcular el KRL utilizando la fórmula científica validada.
- RF-03: El sistema debe mostrar el resultado del cálculo con precisión de 5 decimales.
- RF-04: El sistema debe proporcionar un medidor visual con gradiente de colores que represente el rango del KRL (verde: bajo, amarillo: normal, rojo: alto).
- RF-05: El sistema debe incluir tooltips informativos para cada campo de entrada.
- RF-06: El sistema debe permitir limpiar todos los campos y restablecer el estado inicial.
- RF-07: El sistema debe validar que los datos ingresados sean valores numéricos válidos.
- RF-08: El sistema debe soportar modo claro y oscuro intercambiables.
- RF-09: El sistema debe soportar modo español e inglés intercambiables.

3.3. Requerimientos No Funcionales

- RNF-01 (Usabilidad): La interfaz debe ser intuitiva y no requerir capacitación previa para usuarios con conocimientos básicos de informática.
- RNF-02 (Portabilidad): El sistema debe ser ejecutable en Windows, Linux y macOS sin modificaciones.
- RNF-03 (Rendimiento): El cálculo del KRL debe ejecutarse en menos de 100ms.
- RNF-04 (Precisión): Los cálculos deben mantener precisión de punto flotante de doble precisión.

- RNF-05 (Mantenibilidad): El código debe estar estructurado, comentado y seguir estándares de Python PEP 8.

3.4. Casos de Uso

Actor	Usuario (Científico, Geodesta, Investigador)
Descripción	El usuario ingresa parámetros atmosféricos y obtiene el cálculo del KRL
Precondiciones	La aplicación está abierta y funcional
Flujo Normal	1. El usuario ingresa la altura en metros 2. El usuario ingresa la temperatura en °C 3. El usuario ingresa la presión en hPa 4. El usuario ingresa el gradiente vertical (°C/m) 5. El usuario presiona el botón 'Calcular KRL' 6. El sistema valida los datos 7. El sistema ejecuta el cálculo del KRL 8. El sistema muestra el resultado numérico 9. El sistema actualiza el medidor visual
Flujo Alternativo	6a. Si los datos no son válidos, el sistema muestra mensaje de error
Postcondiciones	El KRL es calculado y mostrado correctamente

Tabla 4: CU-01: Calcular Coeficiente KRL

4. Arquitectura del Sistema

4.1. Arquitectura General

El sistema utiliza una arquitectura de tres capas implementada en un único módulo Python, optimizada para simplicidad y mantenibilidad.

Capa	Responsabilidad	Tecnología
Presentación	Interfaz gráfica de usuario, captura de eventos, visualización de resultados	CustomTkinter, Canvas (Tkinter)
Lógica de Negocio	Implementación de la fórmula KRL, validación de datos, control de flujo	Python (métodos de clase)
Visualización	Renderizado del medidor visual, gradiente de colores, indicador de posición	Canvas (Tkinter), gráficos 2D

Tabla 5: Distribución de responsabilidades

4.2. Stack Tecnológico

Componente	Tecnología/Librería
Lenguaje de Programación	Python 3.8+
Framework GUI	CustomTkinter (interfaz moderna sobre Tkinter)
Componentes UI Adicionales	CTkToolTip (tooltips informativos)
Cálculos Matemáticos	Librería estándar math de Python

Tabla 6: Stack tecnológico

4.3. Componentes del Sistema

Clase Principal: App

La clase `App` hereda de `ctk.CTk` y encapsula toda la lógica de la aplicación. Sus métodos principales son:

- `__init__()`: Constructor que inicializa la interfaz gráfica
- `toggle_mode()`: Alterna entre modo claro y oscuro
- `calcular()`: Ejecuta el cálculo del KRL según la fórmula científica
- `dibujar_gradiente()`: Renderiza el gradiente visual de colores
- `actualizar_medidor()`: Actualiza la posición del indicador en el medidor
- `limpiar()`: Reinicia todos los campos y el estado visual

5. Detalles de Implementación

5.1. Fórmula del Coeficiente KRL

El cálculo del Coeficiente de Refracción Local se implementa mediante la siguiente fórmula científica:

$$KL = 121 \times (P \times 1.3332) / T^2 \times (0.0343 + \partial T / \partial Z)$$

Donde:

- KL: Coeficiente de Refracción Local
- P: Presión atmosférica en hectopascales (hPa)
- T: Temperatura absoluta en Kelvin (K)
- $\partial T / \partial Z$: Gradiente vertical de temperatura (°C/m)

5.2. Validación de Datos

El sistema implementa validación mediante bloque `try-except` que captura errores de conversión numérica (`ValueError`) y muestra mensajes informativos al usuario en caso de datos inválidos.

5.3. Medidor Visual

El medidor visual utiliza un gradiente de colores generado dinámicamente que va de verde (valores bajos de KRL) a amarillo (valores normales) y rojo (valores altos). Los rangos establecidos son:

- **Verde (Bajo):** $KRL \leq 0.11$
- **Amarillo (Normal):** $0.11 < KRL \leq 0.20$
- **Rojo (Alto):** $KRL > 0.20$

6. Pruebas y Validación

6.1. Estrategia de Pruebas

Se implementó una estrategia de pruebas basada en:

- Pruebas Unitarias: Validación de la fórmula matemática con valores conocidos
- Pruebas de Interfaz: Verificación de comportamiento de controles y visualizaciones
- Pruebas de Validación Científica: Comparación de resultados con cálculos manuales del experto de dominio
- Pruebas de Usabilidad: Evaluación de la experiencia del usuario

6.2. Casos de Prueba

ID	Descripción	Entrada	Resultado Esperado
TC-01	Cálculo con valores normales	H=100, T=20, P=1013, G=0.01	KRL calculado con 5 decimales
TC-02	Validación de entrada no numérica	H='abc', T=20, P=1013, G=0.01	Mensaje de error mostrado
TC-03	Función limpiar	Click en botón 'Limpiar'	Todos los campos vacíos, medidor reseteado
TC-04	Cambio de modo claro/oscuro	Toggle del switch	Tema cambia correctamente

Tabla 7: Casos de prueba

7. Instalación y Despliegue

7.1. Requisitos del Sistema

- Sistema Operativo: Windows 10+, Linux (Ubuntu 18.04+), macOS 10.14+
- Python: Versión 3.8 o superior
- RAM: Mínimo 512 MB
- Espacio en disco: 50 MB

7.2. Proceso de Instalación

No necesario al ser un archivo ejecutable

8. Mantenimiento y Soporte

Plan de Mantenimiento

- Mantenimiento Correctivo: Corrección de errores reportados por usuarios
- Mantenimiento Adaptativo: Actualizaciones por cambios en dependencias o sistemas operativos
- Mantenimiento Perfectivo: Mejoras de interfaz y optimizaciones de rendimiento

9. Conclusiones

El sistema Predictor KRL ha sido desarrollado exitosamente utilizando una metodología ágil simplificada, apropiada para el contexto de un equipo reducido. La implementación cumple con los objetivos científicos establecidos por el experto de dominio, proporcionando una herramienta precisa y eficiente para el cálculo del Coeficiente de Refracción Local.

El uso de tecnologías multiplataforma garantiza la accesibilidad del sistema en diversos entornos de trabajo, mientras que la interfaz gráfica intuitiva facilita su adopción por usuarios sin experiencia técnica avanzada.

La arquitectura modular y el código bien documentado aseguran la mantenibilidad futura del sistema, permitiendo incorporar mejoras y adaptaciones según las necesidades que puedan surgir en la comunidad científica.